

How It Fits Together

Inventory, variables, the YAML files and surveys -
and how to update the repository safely

Site Automation teaching guide | site-automation 0.3.5 | September 2026

What this is. How the pieces of site automation fit together: what the code, the inventory, the credentials and the survey each do; how one setting travels from its default in a YAML file to a host; how groups work (and a host in several of them); what a survey is and where to find it; and how to update the repository at work without overwriting the wrong things.

Read this first

This guide explains the ideas. The *Operations Runbooks Setup Guide* has the clicks, one step at a time. The appendix lists every setting you can change.

Rendered from `docs/HOW_IT_FITS_TOGETHER.md` and `roles/*/defaults/main.yml` in the site-automation repository (version 0.3.5).

Contents

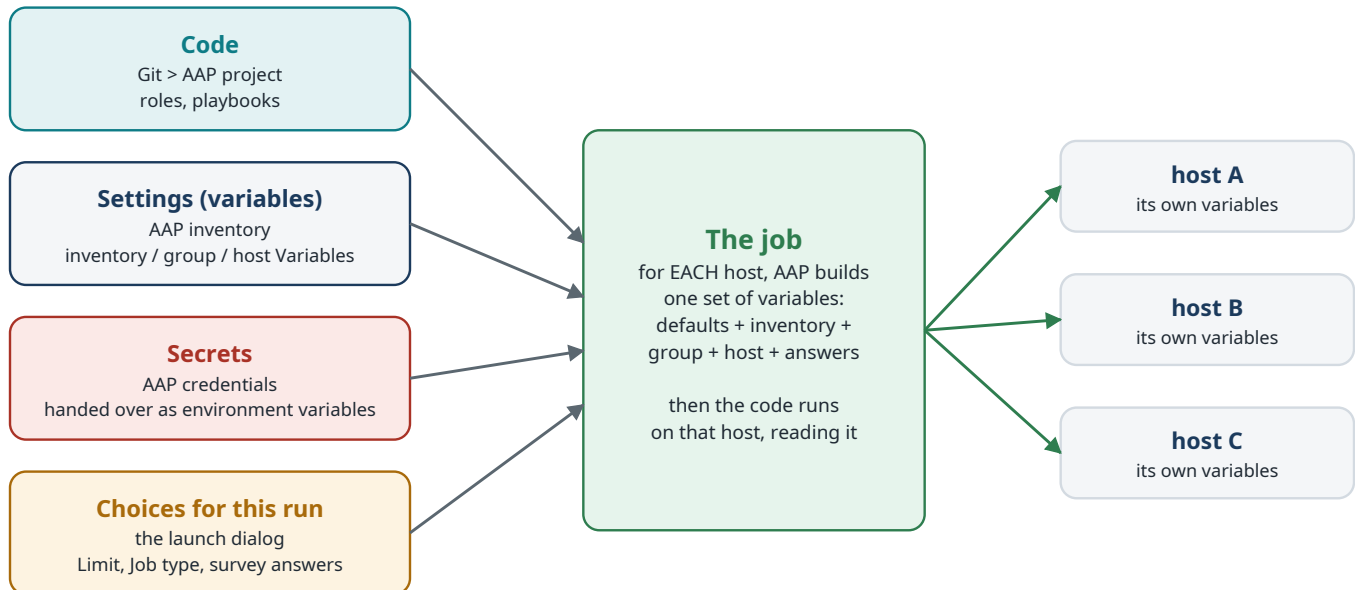
1. How it all fits together	4
The four kinds of pieces	4
Reading YAML in two minutes	5
The YAML files in this repository, and which ones you touch	6
Follow one setting from its default to a host	6
The inventory: one inventory, many groups, and a host in several groups	8
Is it modular? Yes, in three ways	9
Surveys: the questions at launch (run-time variables)	9
Updating the repository at work, without overwriting the wrong things	11
Cheat sheet: where does it go?	13
Appendix: your settings files (playbooks/group_vars/)	14
all.yml (every host)	14
aap.yml (AAP group aap)	15
idm.yml (AAP group idm)	15
logstash.yml (AAP group logstash)	15
mariadb.yml (AAP group mariadb)	16
netapp.yml (AAP group netapp)	16
rhel8_all.yml (AAP group rhel8_all)	16
sat.yml (AAP group sat)	16
stigman.yml (AAP group stigman)	17
Appendix: every setting you can change	18
check_disk (roles/check_disk/defaults/main.yml)	18
check_mounts (roles/check_mounts/defaults/main.yml)	18
check_services (roles/check_services/defaults/main.yml)	19
check_performance (roles/check_performance/defaults/main.yml)	19
check_time (roles/check_time/defaults/main.yml)	19
check_network (roles/check_network/defaults/main.yml)	20
check_logging (roles/check_logging/defaults/main.yml)	20
check_selinux (roles/check_selinux/defaults/main.yml)	20
check_fapolicyd (roles/check_fapolicyd/defaults/main.yml)	21
check_auditd (roles/check_auditd/defaults/main.yml)	21
check_accounts (roles/check_accounts/defaults/main.yml)	21

check_certs (roles/check_certs/defaults/main.yml)	22
check_patching (roles/check_patching/defaults/main.yml)	22
check_mariadb (roles/check_mariadb/defaults/main.yml)	23
site_findings (roles/site_findings/defaults/main.yml)	24
site_act (roles/site_act/defaults/main.yml)	24
servicenow (roles/servicenow/defaults/main.yml)	26
poam (roles/poam/defaults/main.yml)	27
patch (roles/patch/defaults/main.yml)	28
troubleshoot (roles/troubleshoot/defaults/main.yml)	28
service_watch (roles/service_watch/defaults/main.yml)	29
stigman_stack (roles/stigman_stack/defaults/main.yml)	31

1. How it all fits together

This repository has many files, and AAP has many screens. Every one of them does one of **four jobs**. Once you know which job a piece does, you know where to look, and what you may change.

The four kinds of pieces



Four kinds of pieces, each in its own place. AAP combines them into one set of variables per host when a job starts.

Piece	What it holds	Where it lives	Who changes it	Example
Code	<i>what to do</i> , step by step	Git (this repository), seen by AAP as the project	nobody, except to install a new release	<code>roles/check_disk/tasks/main.yml</code>
Settings (variables)	<i>the numbers and names that are different at your site</i> , or for some hosts	your settings files in <code>playbooks/group_vars/</code> (one per AAP group), or the AAP inventory's Variables boxes	you	<code>check_disk_warn_pct: 80</code>
Secrets	passwords, keys, tokens	AAP credentials	you, once	the ServiceNow API password
Choices for this run	which hosts, which checks, dry run or real	the launch dialog: Limit, survey	whoever clicks Launch	<code>health_checks: weekly</code>

When a job starts, AAP puts the four together **for each host**:

1. the code, from the project;
2. the settings, from the inventory. Each host gets the values of its own groups and its own Variables box;
3. the secrets, from the credentials, handed over as environment variables;

4. your answers, from the survey.

The code then reads from that host's combined set of variables. That is the whole machine.

The one rule that keeps upgrades painless: never edit a file that came with a release, except **your** files: your settings in `playbooks/group_vars/` and your POA&M list `poam/poam.csv`. Everything else you want different goes in AAP: secrets in credentials, choices in surveys. A new release then replaces our files, and your settings are untouched.

Reading YAML in two minutes

Every settings box in AAP, and most files here, are YAML: names and values, one per line.

YAML

```
check_disk_warn_pct: 85                # a setting: name, colon, space, value
check_services_required:                # a list: one "- item" per line...
  - sshd
  - crond
check_services_required: [sshd, crond] # ...or the same list on one line
check_network_tcp:                    # a list of small tables (name / host / port each)
  - {name: LDAP, host: ldap.yoursite.mil, port: 636}
check_disk_overrides:                 # a table inside a table
  /var/log/audit: {warn: 70, crit: 85}
```

- **Indentation is spaces, never tabs.** It matters: the things that belong to a name are indented further than the name.
- **#** starts a comment. Everything after it on that line is ignored.
- **{{ something }}** inside a value means "fill in the variable `something` here". You see it in the code; you rarely need it in settings.
- AAP will not save a Variables box whose YAML it cannot read, so a typo shows up right away.

The YAML files in this repository, and which ones you touch

Path	What it is	Edit it?
<code>playbooks/group_vars/*.yaml</code>	your settings , one file per AAP group (<code>mariadb.yaml</code> = group <code>mariadb</code> , <code>all.yaml</code> = every host). Placeholders start with <code>#</code>	yes : this is where you change settings
<code>playbooks/*.yaml</code>	the files AAP runs, one per job template. Each says <i>which hosts</i> and <i>which roles</i>	no
<code>roles/<role>/defaults/main.yaml</code>	every setting of that role , with its default value and a comment saying what it does. The menu of what you can change	no : read it, then set values in AAP
<code>roles/<role>/tasks/*.yaml</code>	the steps. They read the settings	no
<code>roles/<role>/meta/main.yaml</code>	the role's name tag	no
<code>aap/credential_types/*.yaml</code>	forms you paste into AAP (Credential Types)	no
<code>inventories/example/</code>	a sample inventory. Copy blocks from it; AAP does not read it	no
<code>ansible.cfg</code>	tells Ansible where the roles and filters are. AAP uses it too, but ignores its <code>inventory =</code> line	no
<code>vendor/act/</code>	ACT, copied in from its own release	no
<code>docs/, docs/pdf/</code>	these guides	no
<code>tests/, .github/, .ansible-lint, .yamllint</code>	automatic checks of the code	no
<code>poam/poam.csv</code>	your POA&M list	yes

Two places can hold the same setting: your group file in `playbooks/group_vars/`, and AAP's Variables box for that group. **Keep each setting in one place.** If both have it, the file wins. The file is also the easier place: you edit it in VS Code, and Git remembers every change.

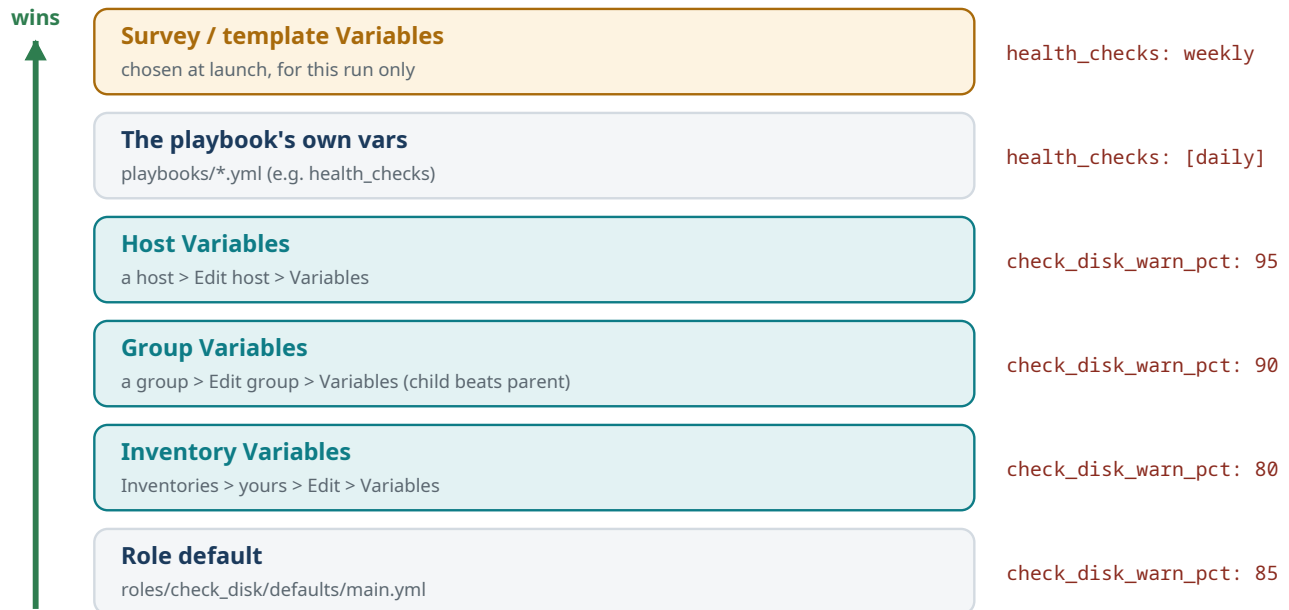
The appendix of this guide prints every `defaults/main.yaml`: every setting there is, in one place.

Follow one setting from its default to a host

Take `check_disk_warn_pct`: warn when a filesystem is this many percent full.

1. **The default.** `roles/check_disk/defaults/main.yaml` says `check_disk_warn_pct: 85`. With nothing else set, every host warns at 85%.
2. **The code reads it.** `roles/check_disk/tasks/main.yaml` compares each filesystem with `check_disk_warn_pct`. You never change this part.
3. **Change it for every host.** Add `check_disk_warn_pct: 80` to `playbooks/group_vars/all.yaml` (or to **Inventories > your inventory > Edit inventory > Variables**). Now every host warns at 80%.

4. **Change it for a group.** Add `check_disk_warn_pct: 90` to the group's file, for example `playbooks/group_vars/mariadb.yml` (or the group's **Edit group > Variables** in AAP). Hosts in that group warn at 90%; all others still at 80%.
5. **Change it for one host.** Open the host > **Edit host > Variables**: `check_disk_warn_pct: 95`. Only that host warns at 95%.
6. **For one run.** A survey answer, or the template's own Variables box, beats all of the above, for that run only. This setting has no survey question, so it simply comes from the inventory.



Where a value can be set, weakest at the bottom; the highest place that sets it wins. A host with its own value gets 95, the other hosts in that group 90, everyone else 80. The top two rows use `health_checks`: it is chosen per run, not per host.

The rule in one line: **the closer to the host, the stronger. An answer given at launch beats everything, for that run.**

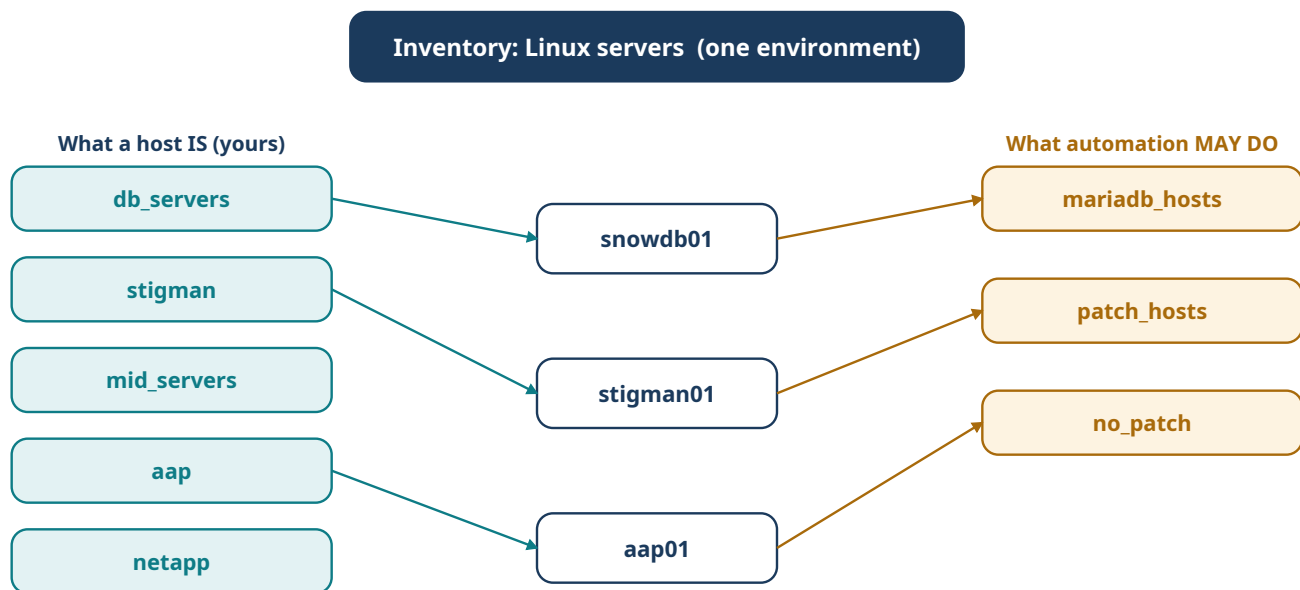
Two things that trip people up:

- **The same setting in two groups of one host.** A host is in both `db_servers` and `stigman`, and both groups set `check_disk_warn_pct`. Ansible then takes the group whose name sorts last alphabetically, which is easy to miss. Set such a value on one group only, or on the host.
- **Some variables are per run, not per host.** Set them at launch (survey) or on the template. For `target` and `health_checks`, an inventory value is even ignored:

Variable	Set it in	Why
<code>target</code> (which hosts a playbook runs on; empty = every host, and <code>stigman</code> for service watch)	the template's Variables box	it picks the hosts <i>before</i> any inventory setting is read
<code>health_checks</code>	the Health check survey	the playbook's own value beats the inventory
<code>ts_area</code> , <code>ts_service</code> , <code>ts_target</code> , <code>ts_since</code>	the Troubleshoot survey	chosen per problem

Variable	Set it in	Why
<code>use_act</code> , <code>site_act_level</code>	a survey (or the inventory, as a fixed default)	chosen per run
everything named <code>check_*</code> , <code>patch_*</code> , <code>site_act_*</code>	your settings files (<code>playbooks/group_vars/</code>), or a group's or host's Variables	facts about your site and hosts. The ACT provider and models are already in <code>all.yml</code> : change them there
<code>servicenow_*</code> , <code>poam_*</code>	<code>playbooks/group_vars/all.yml</code> (the placeholders are there), or the inventory's own Variables box, not both (the file wins). Not a group's box	those jobs run on AAP itself, which is in no group, so it only gets <code>all.yml</code> and inventory-level settings

The inventory: one inventory, many groups, and a host in several groups



Each host is in several groups at once, and gets the Variables of every one of them.

One inventory, many groups. `snowdb01` is in `db_servers` (yours) and also in `mariadb_hosts`, so the MariaDB check runs there. Every host is also in the built-in group `all`: the checks run there when a template has no target.

- **One inventory holds all the hosts of one environment**, for example `Linux servers`. Make a second inventory only for a separate environment (test and production). Never make one to sort hosts by role: that is what groups are for.
- **Groups are labels**. A host can be in as many groups as you like. Adding a host to a group never removes it from its other groups.
- **Groups can contain groups**. Put your `db_servers` group inside `mariadb_hosts` and every database host is in `mariadb_hosts` too, now and whenever you add one later.
- **Settings follow the groups**. A host gets the Variables of every group it is in (and of their parent groups), then its own.

Three kinds of groups work well together, and you already have the first kind:

Kind	Examples	Used for
What a host is	your <code>db_servers</code> , <code>stigman</code> , <code>mid_servers</code> , <code>aap</code> , <code>netapp</code>	settings that belong to that kind of host (the MariaDB container name, the MID keystore path)
What automation may do to it	<code>patch_hosts</code> , <code>no_patch</code> , <code>aap_hosts</code> , <code>netapp_console_hosts</code>	safety: what gets patched, what ACT may only diagnose
Everything	<code>all</code> (built in: you never create it)	where the checks run when a template has no <code>target</code>

Should you make more groups? Yes: a few more **groups**, in your **one** inventory. You have two ways to connect your groups to the names the runbooks use. Both work, so pick one:

- **Way A: add the runbook names as groups, with your groups inside.** A few clicks, and the guides match what you see. For example, create `mariadb_hosts` and put `db_servers` inside it.
- **Way B: tell the runbooks your names.** No new groups. In the inventory **Variables**:

YAML

```
site_act_diagnose_only_groups: [aap, netapp]    # ACT never fixes these
patch_never_reboot_groups: [aap]              # patching never reboots these
patch_never_patch_groups: [aap, netapp]       # patching skips these
```

Then put `target: <the group you patch>` in the Variables box of Patch hosts. (The checks run on every host unless a template sets `target`, for example `target: linux_servers`.) On your database group set `check_mariadb_enabled: true`.

Is it modular? Yes, in three ways

1. **Checks are separate pieces.** Each check is its own role (folder). You pick them by name per run (survey), per schedule, or per workflow step. A check of your own is one new folder (ADDING_ACT.md, "Write your own check").
2. **Settings are separate from code.** Every setting has a default in its role, and you override it at the level that fits: the inventory for everyone, a group for some, a host for one. **It is the same variable name at every level.** So set a value once at the top, and override it only where a group or host is different.
3. **Results are separate from what reads them.** Every check reports findings the same way. The report, ServiceNow tickets and ACT therefore work with every check, including yours.

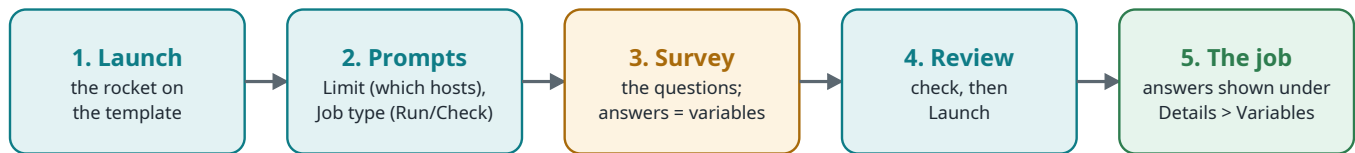
Surveys: the questions at launch (run-time variables)

What it is. The questions a job template asks when someone clicks **Launch**. Each answer becomes a variable for **that run only**. So yes: a survey is how you set variables at run time. It is the safe way: you decide the question, the allowed answers and the default, and people cannot type anything else.

Where it is. **Automation Execution > Templates > open the template > Survey** tab.

- **Create survey question** adds one. **Question** is what people read. **Answer variable name** is the variable it sets (for example `health_checks`). **Answer type** is, for example, *Text*, *Integer*, *Multiple Choice (single select)*, *Multiple Choice (multiple select)*. Then the choices, a **Default answer**, and **Required**.
- **The switch at the top of the tab** turns the survey on. A survey that is off is never asked.

At launch:



Schedules and workflow steps skip 2-4: their answers are set once (Prompts / Survey step).

What happens when you click Launch. The survey is where you set variables for one run.

1. Click the rocket (**Launch**).
2. AAP first asks anything the template "prompts on launch" for, such as the **Limit** (which hosts) or the **Job type** (Run, or Check for a dry run).
3. Then the **survey** page: answer, or keep the defaults.
4. **Next > Launch**. The job's **Details** tab shows the answers under **Variables**. So you can always see what a run was asked to do.

In schedules and workflows there is nobody to answer. So you give the answers once: in the schedule's **Prompts** step, or in the workflow step's **Survey** step. Without them, the **defaults** are used.

Survey, template Variables, or inventory Variables?

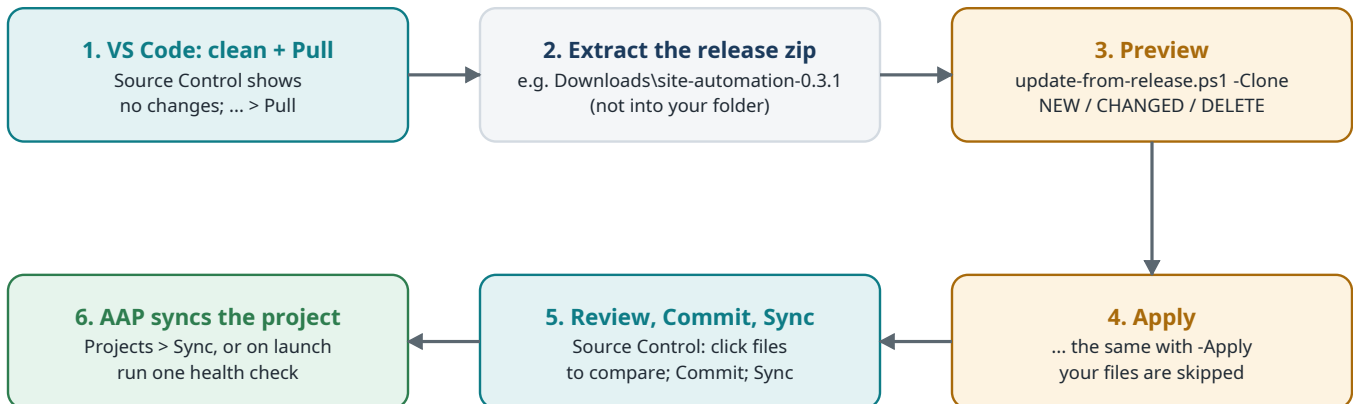
	Survey	Template Variables box	Inventory / group / host Variables
Set by	whoever launches	whoever owns the template	whoever owns the inventory
Applies to	this run	every run of this template	every run, on those hosts
Example	<code>health_checks: weekly</code>	<code>target: linux_servers</code>	<code>check_disk_warn_pct: 80</code>

Never put a password in a survey or a Variables box. Secrets go in **credentials**.

The survey questions these runbooks use:

Template	Variable	Answers
Health check	<code>health_checks</code> (multiple select)	<code>daily</code> , <code>weekly</code> , <code>all</code> , or check names
Troubleshoot	<code>ts_area</code> , <code>ts_service</code> , <code>ts_target</code> , <code>ts_since</code>	area; unit name; host:port; how far back
Certificate report	<code>check_certs_warn_days</code>	days (default 30)
POA&M status	<code>poam_stigman_api</code>	STIG Manager API URL, or blank
Patch hosts	<code>patch_security_only</code> , <code>patch_reboot</code>	<code>false/true</code> ; <code>when_needed/never</code>
(with ACT) Health check, Troubleshoot	<code>use_act</code> , <code>site_act_level</code> , <code>site_act_provider</code> , <code>site_act_model</code>	<code>no/yes</code> ; <code>explain/diagnose/self-heal</code> ; <code>genai/asksage/genai-beta</code> ; a model id

Updating the repository at work, without overwriting the wrong things



Yours, never touched: poam.csv, inventories\site, .site-local and every path it lists.

Updating the work repository from Windows. Nothing reaches your Git server (or AAP) until you click Sync Changes, after you have seen every change.

Think of your work repository as **our files + your few files**. A new release replaces our files and never touches yours. Your files are:

- `playbooks/group_vars/` and `playbooks/host_vars/` (your settings files). A new release may add a settings file you do not have yet; it never changes one you have;
- `poam/poam.csv` (your POA&M list);
- `inventories/site/`, only if you ever keep an inventory in Git (you keep yours in AAP);
- anything you added yourself, such as your own check role. List those paths in a file called `.site-local` at the top of your repository folder, one per line, for example `roles/check_tmp/`.

These steps are for **Windows with VS Code**: your repository folder is open in VS Code, and VS Code does the Git part (commit, push). Nothing here needs Git on the RHEL servers. AAP pulls from your Git server by itself.

Method 1: the update script (recommended)

1. **Get your folder up to date.** In VS Code, open your site-automation folder (**File > Open Folder**). Open **Source Control** (Ctrl+Shift+G): it must show **no changes**. Commit or discard anything listed there first. Then ... > **Pull**, to get whatever is newest on the server.
2. **Extract the new release somewhere else.** Right-click `site-automation-0.3.1.zip` > **Extract All** > for example `Downloads\site-automation-0.3.1`. **Not** into your repository folder.
3. **Preview.** It changes nothing. **Terminal > New Terminal** (it is PowerShell), then:

```
$rel = "$HOME\Downloads\site-automation-0.3.1"          # where you extracted the release
& "$rel\scripts\update-from-release.ps1" -Clone "C:\git\site-automation"
```

`-Clone` is your repository folder: the one VS Code has open. The script lists every file as **NEW**, **CHANGED**, **DELETE**, or **YOURS** (a settings file you do not have yet: added once, then yours). **Read the DELETE lines.** If one is yours, add its path to `.site-local`, commit that, and preview again.

4. **Apply.** The same command, with `-Apply` at the end:

```
& "$rel\scripts\update-from-release.ps1" -Clone "C:\git\site-automation" -Apply
```

5. **Review in VS Code. Source Control** now lists every file the update changed. Click one to see the old and new versions side by side. See something of yours? Right-click it › **Discard Changes** keeps your version.
6. **Commit and push.** Type a message (`site-automation 0.3.1`), click **Commit**, and answer Yes if VS Code asks to stage all changes. Then click **Sync Changes**. Only now does the new version reach your Git server.
7. **In AAP: Projects › site-automation › Sync.** With **Update revision on launch** ticked, the next job syncs by itself. The project page shows the new revision (the commit id).
8. **Check:** run **Health check** on one host.

Changed your mind before step 6? Source Control › ... › Changes › Discard All Changes. Nothing left your PC.

"Running scripts is disabled" or "not digitally signed" when you run the script? Run it through a new PowerShell that is allowed to run it this one time (add `-Apply` for step 4):

```
cd $rel
powershell -ExecutionPolicy Bypass -File .\scripts\update-from-release.ps1 -Clone
"C:\git\site-automation"
```

If your organization blocks that too, use Method 2.

Method 2: copy by hand (when you cannot run scripts)

1. Steps 1 and 2 above.
2. In the **extracted** folder, delete `poam\poam.csv` and anything else you have your own version of. They then cannot overwrite yours.
3. Select everything in the extracted folder › **Copy** › go to your repository folder › **Paste** › **Replace the files in the destination.**
4. Steps 5 to 8 above. Copying never deletes anything, so a file the new release *removed* stays in your folder. The `CHANGELOG.md` says when a release removes one; delete it by hand.

Line endings are taken care of. The repository's `.gitattributes` keeps Linux line endings (LF) in Git, even when you edit and commit on Windows. That matters: the playbooks run shell commands on RHEL, and those break with Windows line endings. VS Code shows `LF` in its status bar; leave it that way.

On Linux instead of Windows? `scripts/update-from-release.sh` does the same as Method 1: `/tmp/site-automation-0.3.1/scripts/update-from-release.sh ~/site-automation`, then again with `--apply`, then `git status`, `git commit`, `git push`.

First time, with no repository at work yet? Create an empty repository on your Git server, clone it in VS Code (**Source Control › Clone Repository**), copy the extracted release into that folder, then **Commit** and **Publish/Sync**. `SETUP_AAP.md` step 1 covers the inventory part.

Cheat sheet: where does it go?

You want to...	Put it in
add a server	AAP inventory › Hosts (and into its groups)
say what a server is (database, STIG Manager, MID...)	AAP inventory › Groups
change a threshold or name for every host	<code>playbooks/group_vars/all.yml</code> (or inventory › Edit inventory › Variables)
...for some hosts	the group's file, <code>playbooks/group_vars/<group>.yml</code> (or the group › Edit group › Variables)
...for one host	the host › Edit host › Variables
store a password or key	Credentials
let people choose at launch	the template's Survey
make a template always run on your group	the template's Variables box: <code>target: <group></code>
pick which checks a schedule runs	the schedule's Prompts (survey answers)
update the code	a new release, with <code>scripts/update-from-release.ps1</code> in VS Code (above)
change what a check does	a new check of your own (ADDING_ACT.md); never edit ours

Appendix: your settings files (playbooks/group_vars/)

One file per AAP group; `all.yml` is every host. Lines starting with `#` are off: delete the `#` and replace `CHANGE-ME` to turn a placeholder on. Edit them in VS Code, Commit, Sync Changes - AAP uses them on the next run. These are YOUR files: updates never change them.

all.yml (every host)

YAML

```
# YOUR SETTINGS FOR EVERY HOST - and for the jobs that run on AAP itself (ServiceNow tickets,
# ServiceNow health, POA&M status).
#
# To change a setting: edit this file in VS Code, Commit, Sync Changes. AAP uses it on the next
# run (the project syncs itself when "Update revision on launch" is ticked).
# A line starting with "#" is OFF. To turn a placeholder on: delete the "#" in front of it (and
  in
# front of the lines that belong to it) and replace CHANGE-ME with your value. Keep the
# indentation exactly as shown: spaces, never tabs.
#
# The files in this folder are YOURS: the update script adds them the first time and never
# changes them again. File name = AAP group name (mariadb.yml -> the hosts in group "mariadb").
# What each setting does: roles/<role>/defaults/main.yml. Keep each setting in ONE place - here,
# or in AAP's Variables boxes (docs/USING_YOUR_AAP_INVENTORY.md) - not both.

# ---- Safety: hosts automation must be careful with (your AAP group names) -----
# ACT may only diagnose these; it never fixes anything on them by itself.
site_act_diagnose_only_groups: [aap, sat, idm, netapp, aap_hosts, netapp_console_hosts]
# Patching skips these completely (Satellite is upgraded with satellite-maintain, not dnf).
patch_never_patch_groups: [aap, sat, netapp, no_patch]
# Patching never reboots these.
patch_never_reboot_groups: [aap, sat, netapp, aap_hosts]

# ---- ACT: which provider, and the model for each -----
site_act_provider: genai          # the default provider: genai | asksage | genai-beta
site_act_models:                 # the model follows whichever provider is picked
  genai: gemini-3.8-flash
  asksage: gpt-5.6-sol-gov
# Ask Sage URL (empty = ACT's default,
  https://api.genai.army.mil/server/openai/v1/chat/completions):
# site_act_urls:
#   asksage: https://CHANGE-ME/server/openai/v1/chat/completions

# ---- What every host must be able to reach -----
```

YAML (continued)

```
# check_network_tcp:
#   - {name: IdM LDAPS, host: CHANGE-ME.yoursite.mil, port: 636}
# check_logging_remote:
#   - {name: logstash, host: CHANGE-ME.yoursite.mil, port: 514}

# ---- The account AAP logs in as -----
# check_accounts_nopasswd_allowed: [CHANGE-ME-service-account]
# check_accounts_expiry_watch: [CHANGE-ME-service-account]

# ---- ServiceNow tickets -----
# servicenow_assignment_group: CHANGE-ME          # a group that exists in ServiceNow
# servicenow_min_severity: warning                # or critical: tickets for critical findings
# only

# ---- POA&M: STIG Manager cross-check (optional)
# -----
# poam_stigman_api: https://CHANGE-ME/api
```

aap.yml (AAP group aap)

YAML

```
# YOUR SETTINGS FOR THE AAP HOSTS (AAP group aap). ACT never fixes anything here, and patching
# skips these hosts (all.yml).
check_disk_overrides:
  /var: {warn: 80, crit: 90}          # AAP keeps job output and container images under /var
check_certs_endpoints:
  - {name: AAP web UI, host: 127.0.0.1, port: 443}
```

idm.yml (AAP group idm)

YAML

```
# YOUR SETTINGS FOR THE IDM SERVERS (AAP group idm). ACT never fixes anything here (all.yml).
check_services_required_extra: [ipa]      # the IdM services, as one unit (systemctl status ipa)
check_certs_endpoints:                   # IdM's certificates expiring breaks every login
  - {name: IdM web UI, host: 127.0.0.1, port: 443}
  - {name: IdM LDAPS, host: 127.0.0.1, port: 636}
```

logstash.yml (AAP group logstash)

YAML

```
# YOUR SETTINGS FOR THE LOGSTASH HOSTS (AAP group logstash).
check_services_required_extra: [logstash]
```

mariadb.yml (AAP group mariadb)

YAML

```
# YOUR SETTINGS FOR THE MARIADB HOSTS (AAP group mariadb). MariaDB runs in a podman container.
# To turn the MariaDB check on:
#   1. On a MariaDB host, run: sudo podman ps - and note the MariaDB container's NAME.
#   2. Below, delete the "# " in front of BOTH lines and replace CHANGE-ME with that name.
#   3. In AAP, attach the "MariaDB monitor" credential (the read-only DB account) to the Health
#       check and Troubleshoot job templates. Credentials attach to templates, not to groups; the
#       check only uses it on the hosts in this group.
# check_mariadb_enabled: true
# check_mariadb_container: CHANGE-ME
```

netapp.yml (AAP group netapp)

YAML

```
# YOUR SETTINGS FOR THE NETAPP HOSTS (AAP group netapp).
# Nothing is needed yet: all.yml already makes these hosts diagnose-only for ACT and skipped by
# patching (vendor-managed). They are checked like any other RHEL host.
```

rhel8_all.yml (AAP group rhel8_all)

YAML

```
# YOUR SETTINGS FOR THE RHEL 8 HOSTS (AAP group rhel8_all).
# Only needed if AAP's Ansible is 2.17 or newer: it cannot use RHEL 8's built-in Python 3.6.
# Install
# python39 on these hosts (from Satellite), then delete the "# " below. ACT also needs Python
# 3.8+.
# ansible_python_interpreter: /usr/bin/python3.9
```

sat.yml (AAP group sat)

YAML

```
# YOUR SETTINGS FOR THE SATELLITE HOSTS (AAP group sat). ACT never fixes anything here, and
# patching skips these hosts: Satellite is upgraded with satellite-maintain (all.yml).
check_disk_overrides:
  /var: {warn: 80, crit: 90}          # Satellite keeps its content (Pulp) under /var
check_certs_endpoints:
  - {name: Satellite web UI, host: 127.0.0.1, port: 443}
# Services that must run. Check the names first: satellite-maintain service list
# check_services_required_extra: [foreman, httpd, postgresql, pulpcore-api, pulpcore-content]
```


stigman.yml (AAP group stigman)

YAML

```
# YOUR SETTINGS FOR THE STIG MANAGER HOSTS (AAP group stigman).
# Service watch and STIG Manager deploy run on this group by themselves (no target needed).
# Service watch: the containers to watch, dependencies first. A container name (sudo podman ps -a)
# or a service name without .service: how each one is run (systemd unit or plain podman) is found
# by itself. Add `unit: NAME.service` under one only if the job output shows the wrong unit.
watch_containers:
  - name: mysql
  - name: kcmysql
  - name: keycloak
  - name: stigman
  - name: nginx
check_certs_endpoints:          # the certificate browsers see
  - {name: STIG Manager, host: 127.0.0.1, port: 443}
# check_certs_files: [/etc/CHANGE-ME/*.crt] # also check the certificate files, if you like
```

Appendix: every setting you can change

Each block is one role's `defaults/main.yml`: the settings with their default values and what they do. To change one, set the same name in AAP (inventory, group or host Variables) - never edit the file. Names that start with an underscore are internal.

check_disk (roles/check_disk/defaults/main.yml)

YAML

```
# DISK: how full each LOCAL filesystem is (space and inodes), and - from the history of earlier
# runs - how many days until it is full. Network filesystems (NFS) are the mounts check's job:
# a hung NFS server can make `df` hang, so this check never touches them.

check_disk_warn_pct: 85                # % used -> warning
check_disk_crit_pct: 95                # % used -> critical
check_disk_inode_warn_pct: 85          # inodes: too many small files fill a disk too
check_disk_inode_crit_pct: 95

# Different limits for particular mount points, e.g. keep the audit log partition emptier:
# check_disk_overrides:
#   /var/log/audit: {warn: 70, crit: 85}
check_disk_overrides: {}

# Mount points to skip (regular expressions).
check_disk_ignore:
  - '^/(proc|sys|dev|run)(/|$)'
  - '^/etc/(hosts|hostname|resolv\.conf)$' # bind-mounted files (containers)
  - '^/var/lib/containers/storage/'
check_disk_exclude_types: [tmpfs, devtmpfs, squashfs, overlay, iso9660, efivarfs, nsfs]

# Days-until-full forecast. Each run adds a sample (date, bytes used) per mount to a small file
# on the host; once there is at least a day of history, the growth rate predicts when the
# filesystem fills. Check mode (a dry run) reads the history but does not add to it.
check_disk_forecast: true
check_disk_forecast_warn_days: 14
check_disk_forecast_crit_days: 3
check_disk_history_file: /var/lib/site-health/disk-history.json
check_disk_history_keep_days: 30

check_disk_timeout: 20                # seconds for df
```

check_mounts (roles/check_mounts/defaults/main.yml)

YAML

```
# MOUNTS: filesystems that should be mounted but are not, local filesystems that went
# read-only (the kernel does that after disk errors), and network mounts (NFS/CIFS) that hang.
# Diagnose only: this check never mounts, unmounts or remounts anything.

check_mounts_network_types: [nfs, nfs4, cifs, smb3]
check_mounts_local_types: [xfs, ext4, ext3, btrfs, vfat]
check_mounts_timeout: 5               # seconds before a network mount counts as hung
# Network mounts: how full (same idea as the disk check)
check_mounts_warn_pct: 85
check_mounts_crit_pct: 95
# fstab mount points that are allowed to be unmounted (regular expressions)
check_mounts_ignore: []
```

check_services (roles/check_services/defaults/main.yml)

YAML

```
# SERVICES: systemd units that failed, services that must be running (and start at boot), and
# services stuck restarting over and over.

# Must be running on every host. A host where one is not installed gets a warning (set
# check_services_required in the inventory to change the list for a group of hosts).
check_services_required: [sshd, crond, rsyslog, chronyd, auditd]
# More for a group of hosts, e.g. in group_vars/mariadb_hosts.yml: [mariadb]
check_services_required_extra: []
# Failed units to ignore (regular expressions), e.g. a known-broken vendor unit.
check_services_ignore_failed: []
# More restarts than this since the unit started = a restart loop
check_services_restart_warn: 3
```

check_performance (roles/check_performance/defaults/main.yml)

YAML

```
# PERFORMANCE: is the host overloaded right now, or was something killed for lack of memory?

check_performance_load_warn: 1.5          # 15-minute load average PER CPU (1.0 = every CPU busy)
check_performance_load_crit: 3.0
check_performance_mem_avail_warn_pct: 10  # memory still available for programs, % of total
check_performance_mem_avail_crit_pct: 5
check_performance_swap_warn_pct: 50       # swap in use, % of swap
check_performance_swap_crit_pct: 80
check_performance_iowait_warn: 20         # % of CPU time waiting for disk (average over the
sample)
check_performance_iowait_crit: 40
check_performance_steal_warn: 10          # % of CPU taken by the hypervisor (VM starved)
check_performance_sample_seconds: 5       # how long vmstat samples
check_performance_oom_since: "-24h"       # look for out-of-memory kills this far back
check_performance_zombie_warn: 20
```

check_time (roles/check_time/defaults/main.yml)

YAML

```
# TIME: is the clock synchronized, and by how much is it off? A clock that is off by minutes
# breaks Kerberos/AD logins (5 minutes), TLS, log correlation and audit evidence.

check_time_offset_warn: 0.5              # seconds
check_time_offset_crit: 5.0
check_time_min_sources: 1                # time servers chrony must be able to reach
```

check_network (roles/check_network/defaults/main.yml)

YAML

```
# NETWORK: can this host reach what it depends on? A default route, names that must resolve,
# and TCP ports that must answer (from THIS host, which is what matters when a service breaks).

check_network_require_default_route: true
check_network_resolve_self: true          # this host's own full name (hostname -f) resolves
check_network_dns_names: []              # e.g. [ldap.example.mil, servicenow.example.mil]
# TCP services this host needs, e.g.
#   check_network_tcp:
#     - {name: LDAP, host: ldap.example.mil, port: 636}
#     - {name: SIEM (syslog over TLS), host: siem.example.mil, port: 6514}
check_network_tcp: []
check_network_timeout: 5
```

check_logging (roles/check_logging/defaults/main.yml)

YAML

```
# LOGGING: are logs kept, and do they reach the central log server (SIEM)?

check_logging_journal_persistent: true    # the journal must survive a reboot (/var/log/journal)
# The central log server(s) this host forwards to, e.g.
#   check_logging_remote:
#     - {name: SIEM, host: siem.example.mil, port: 6514}
check_logging_remote: []
check_logging_timeout: 5
# rsyslog keeps messages it cannot deliver in /var/lib/rsyslog. A big queue = the log server
# has been unreachable for a while.
check_logging_queue_warn_mb: 100
# Error-priority journal messages in the last day. Many = something is complaining a lot.
check_logging_errors_since: "-24h"
check_logging_errors_warn: 500
```

check_selinux (roles/check_selinux/defaults/main.yml)

YAML

```
# SELINUX: enforcing (the STIG requires it), the config file agrees (so a reboot does not
# change it), no locally added permissive domains, and recent denials (AVCs) - which is what
# you see when SELinux blocks an application.

check_selinux_expected_mode: enforcing
check_selinux_avc_since: yesterday        # ausearch -ts: recent (10 min), today, yesterday,
this-week
check_selinux_avc_warn: 1                  # this many denials or more -> warning
# Booleans that must have a value, e.g. {httpd_can_network_connect: "on"}
check_selinux_booleans: {}
# Paths whose file labels are compared with the policy (a dry-run restorecon). Mislabeled files
# are a common cause of denials after files are copied or moved. Keep it to small trees.
check_selinux_relabel_paths: []           # e.g. [/etc/nginx, /var/www, /etc/stigman]
```

check_fapolicyd (roles/check_fapolicyd/defaults/main.yml)

YAML

```
# FAPOLICYD (application allow-listing): running, enforcing, configuration and trust database
# valid, and recent denials - which is what you see when it blocks a program.

# true = every host in the play must have fapolicyd. false = only check hosts that have it.
check_fapolicyd_required: false
check_fapolicyd_expect_enforcing: true # permissive = 1 means it only logs, never blocks
# Compare the trust database with the files on disk (can take a minute on big hosts; files
# changed outside RPM show up here, e.g. after a manual update).
check_fapolicyd_check_trustdb: true
check_fapolicyd_denials_since: yesterday # ausearch -ts value
check_fapolicyd_denials_warn: 1
```

check_auditd (roles/check_auditd/defaults/main.yml)

YAML

```
# AUDITD: the audit system records who did what (the STIG depends on it). Running, enabled in
# the kernel, rules loaded, no events lost, room for its log, and (optionally) forwarding.

check_auditd_min_rules: 5 # fewer loaded rules than this = the STIG rules are
missing
check_auditd_log_warn_pct: 75 # the audit log's filesystem
check_auditd_log_crit_pct: 90
check_auditd_remote_required: false # true = au-remote (audisp-remote) must be active
```

check_accounts (roles/check_accounts/defaults/main.yml)

YAML

```
# ACCOUNTS: the account problems auditors and attackers both look for, plus the one that
# quietly breaks automation - a service account password about to expire.

check_accounts_min_uid: 1000 # people's accounts start here
check_accounts_inactive_days: 35 # the STIG wants unused accounts disabled after 35 days
# Accounts that may stay unused (break-glass, service accounts that never log in interactively)
check_accounts_ignore_inactive: []
# sudoers entries allowed to use NOPASSWD, e.g. the account AAP connects as: [svc_aap, "%wheel"]
check_accounts_nopasswd_allowed: []
# Accounts whose password/account expiry matters - above all the account AAP logs in with.
check_accounts_expiry_watch: [] # e.g. [svc_aap]
check_accounts_expiry_warn_days: 14
```

check_certs (roles/check_certs/defaults/main.yml)

YAML

```
# CERTS: certificates that expire soon - as files on the host, inside Java keystores, and as
# served by TLS endpoints (what a browser or client actually sees).

check_certs_warn_days: 30
check_certs_crit_days: 14

# Certificate files (PEM) to check. Globs work. Every certificate in a file is checked (a chain
# file with an expiring intermediate is a problem too).
check_certs_files: [] # e.g. [/etc/stigman/*.crt,
                      /etc/pki/tls/certs/server.crt]

# Also look for certificates in the usual places. Files with many certificates are CA bundles
# (hundreds of roots, noisy) and are skipped unless listed in check_certs_files.
check_certs_discover: true
check_certs_discover_dirs: [/etc/pki/tls/certs, /etc/pki/nginx, /etc/nginx, /etc/httpd/conf.d,
                           /etc/stigman]
check_certs_discover_exclude: 'ca-bundle|ca-certificates|/localhost\.crt$'
check_certs_discover_max_per_file: 10

# TLS endpoints, checked FROM this host, e.g. the site's own web server:
#   check_certs_endpoints:
#     - {name: STIG Manager, host: 127.0.0.1, port: 443, servername: stigman.example.mil}
check_certs_endpoints: []

# Java keystores (Tomcat, the ServiceNow MID Server ...). A JKS keystore lists without a
# password; a PKCS12 one needs it - give the NAME of an environment variable that holds it
# (set by an AAP credential), never the password itself:
#   check_certs_keystores:
#     - {path: /opt/servicenow/mid/agent/security/agent_keystore, password_env: KEYSTORE_PASSWORD}
check_certs_keystores: []
```

check_patching (roles/check_patching/defaults/main.yml)

YAML

```
# PATCHING (readiness, read-only): pending security updates, a reboot still needed after the
# last updates, and how long ago the host was last updated. Installs nothing - patching itself
# is playbooks/patch_hosts.yml.

check_patching_max_age_days: 35 # last package update older than this -> warning
check_patching_security_warn: 1 # this many pending security advisories -> warning
check_patching_critical_is_critical: true # a pending Critical-severity advisory -> critical
check_patching_timeout: 300 # seconds for dnf to talk to the repositories
check_patching_reboot_check: true
```

check_mariadb (roles/check_mariadb/defaults/main.yml)

YAML

```
# MARIADB health (read-only): running, accepting connections, connection headroom, queries that
# run too long, replication / Galera state, recent errors in its log, and room on its disk.
# It runs status queries only - no data is read and nothing is changed.

# Which hosts: members of the mariadb_hosts inventory group (or set this true for a host).
check_mariadb_enabled: "{{ 'mariadb_hosts' in group_names }}"

# MariaDB in a podman container (e.g. the ServiceNow database): the container NAME. The client
# inside the container is used (podman exec). Empty = MariaDB is installed on the host itself.
check_mariadb_container: ""
# The systemd unit (host installs). Empty = the first of mariadb, mysqld, mysql that exists.
check_mariadb_service: ""

# How to log in. Without a credential: as root through the local socket - MariaDB's default for
# a host install on RHEL (no password needed when the job runs with sudo). MariaDB in a
# CONTAINER usually gives root a password (the official images do), so there attach the
# "MariaDB monitor" AAP credential: it sets MARIADB_MONITOR_USER / MARIADB_MONITOR_PASSWORD, and
# the password reaches the client only through the environment (never the command line).
# Extra client options, e.g. ['--socket=/var/lib/mysql/mysql.sock'] or ['--host=127.0.0.1',
# '--port=3306']
check_mariadb_client_args: []

check_mariadb_conn_warn_pct: 80          # connections in use, % of max_connections
check_mariadb_conn_crit_pct: 95
check_mariadb_threads_running_warn: 32   # queries executing at this moment
check_mariadb_long_query_seconds: 300    # a query running longer than this is "long"
check_mariadb_replica_lag_warn: 300       # seconds a replica is behind its primary
check_mariadb_replica_lag_crit: 1800
check_mariadb_galera_size: 0              # expected Galera cluster size (0 = not Galera)
check_mariadb_restart_warn_minutes: 60    # restarted less than this ago -> warning
check_mariadb_bufferpool_hit_warn: 95     # % of reads served from memory (only judged after 1M
reads)
check_mariadb_error_since_days: 1         # [ERROR] lines in its log this recent
check_mariadb_disk_warn_pct: 85          # the filesystem holding the data directory
```

YAML (continued)

```
check_mariadb_disk_crit_pct: 95
```

site_findings (roles/site_findings/defaults/main.yml)

YAML

```
# Shared by every check playbook. A "finding" is one thing that is wrong on one host:
#
#   check:    the check that found it (disk, services, mariadb, ...)
#   id:       a stable name for this problem on this host, e.g. disk:/var. ServiceNow uses it to
#             update the same ticket instead of opening a new one every run.
#   severity: critical | warning
#   summary:  one line a person understands ("/var is 96% full (critical at 95%)")
#   hint:     a read-only command that shows more. A person can paste it into a shell; ACT (when
#             you add it later) runs it to collect evidence.
#
# Check roles only ADD findings. This role starts the list, reports it, and decides pass/fail.

# Which severities make the job FAIL. A failed job is what sends an AAP workflow down its
# "on failure" path (open tickets, ask ACT, wait for approval). [] = report only, never fail.
site_fail_on: [critical, warning]

# One syslog line per finding (tag below), so your log server / SIEM sees them too.
site_syslog: true
site_syslog_tag: site-health
```

site_act (roles/site_act/defaults/main.yml)

YAML

```
# ACT (GenAI) on top of ANY check playbook. The check roles already did the cheap, certain part:
# they found WHAT is wrong (site_findings) and know which read-only command shows more (each
# finding's `hint`). This role hands that to ACT (the act_triage role) to find out WHY, and to
# propose - or, if you allow it, make - the fix. Turn it on with use_act=true.
#
# Pick one level per job template:
#   explain  The role runs the findings' look commands itself and gives ACT their output.
#            ACT runs NOTHING. You get a root cause and a suggested fix in words.
#            Safest start; no approval step needed. (ACT evidence mode)
#   diagnose ACT investigates by running read-only commands itself and proposes exact fix
#            commands. Every change is refused and waits for a person: the job fails with
#            NEEDS APPROVAL, the workflow's approval step asks, and act_fix_approved.yml
#            applies exactly what was approved. (ACT loop mode, no
--allow)
#   self-heal Like diagnose, but ACT may run the fixes that match site_act_allow by itself
#            (e.g. restart chronyd). Anything else still waits for approval.
site_act_level: explain

# self-heal only: regular expressions of fix commands ACT may run without a person. Each must
# match the WHOLE command. Keep them narrow, e.g.
#   site_act_allow: ['systemctl restart (chronyd|rsyslog|crond)']
site_act_allow: []

# Hosts where ACT may only diagnose - never fix by itself, whatever site_act_level says: AAP
# itself (a fix could stop the job that is running it) and vendor-managed appliances. The apply
# step (act_fix_approved.yml) skips them too: a person applies the fix there by hand.
site_act_diagnose_only_groups: [aap_hosts, netapp_console_hosts]

# ---- Which model ACT talks to -----
#   site_act_provider  genai      GenAI.mil (the default)
#                     asksage    Ask Sage (set site_act_url to your organization's instance)
#                     genai-beta the GenAI.mil beta proxy (preview models)
#   site_act_models    the model to use FOR EACH provider, e.g.
#                     {genai: gemini-3.8-flash, asksage: gpt-5.6-sol-gov}
```


YAML (continued)

```

#           so the model follows whichever provider is picked. A provider not listed
#           uses ACT's own default model.
#   site_act_urls   the API URL for each provider, e.g. {asksage:
#             https://<host>/server/openai/v1/chat/completions};
#           a provider not listed uses ACT's default URL
#   site_act_model  one model for this run, whatever the provider (e.g. a survey answer);
#           empty = site_act_models
#   site_act_url    one URL for this run; empty = site_act_urls
#   site_act_ca     a CA bundle file ON THE HOSTS for the provider's TLS certificate
#             (internal/DoD
#             CA); empty = the host's system trust store
# The API KEY never goes here: it comes from an AAP credential - "ACT model key" (GenAI and Ask
# Sage keys) or "ACT GenAI beta key". ACT runs ON EACH HOST, so the hosts must be able to reach
# the provider's URL (a proxy or firewall rule may be needed).
site_act_provider: genai
site_act_models: {}
site_act_urls: {}
site_act_model: ""
site_act_url: ""
site_act_ca: ""

# Any other ACT environment settings (advanced; these win over the ones above), e.g.
# {GENAI_TIMEOUT: "180"}. See the top of vendor/act/act for the list.
site_act_env: {}
site_act_timeout: 600           # seconds for one ACT run
site_act_extra_instructions: "" # anything ACT should know about this site
site_act_evidence_text: ""      # extra evidence (troubleshoot.yml passes what it
                                collected)

```

servicenow (roles/servicenow/defaults/main.yml)

YAML

```
# SERVICENOW - open, update (and optionally resolve) incidents from check findings, and check
# that ServiceNow itself (the instance and its MID Servers) is healthy. Uses the REST Table API
# with ansible.builtin.uri: no extra collection to install.
#
# Connection: the "ServiceNow API" AAP credential sets SN_HOST (https://<instance>), SN_USERNAME
# and SN_PASSWORD in the job's environment. Without it, nothing is sent: the job prints what it
# WOULD have opened (a safe way to try it out).

# Findings at or above this severity get a ticket: critical | warning
servicenow_min_severity: warning

servicenow_table: incident
# Values for new incidents. Names work (the API is asked to accept display values), e.g.
# servicenow_assignment_group: Linux Operations
servicenow_assignment_group: ""
servicenow_caller: "" # user name; empty = the API account
servicenow_category: "" # e.g. Software
servicenow_urgency: {critical: "1", warning: "2"}
servicenow_impact: {critical: "2", warning: "3"}
servicenow_set_ci: true # set Configuration item = the host name (must exist in
the CMDB)

# When a problem that had a ticket is gone, add a work note (and, if you want, resolve it).
servicenow_note_cleared: true
servicenow_resolve_cleared: false
servicenow_resolved_state: "6" # Resolved
servicenow_close_code: "Solution provided" # must be a close code your instance has
# Add a work note to an open ticket each time its finding is seen again.
servicenow_note_repeats: true

# Every ticket from these playbooks carries a correlation ID starting with this, so the same
# problem on the same host always updates the same ticket.
servicenow_correlation_prefix: site-health
```

YAML (continued)

```
servicenow_validate_certs: true
servicenow_timeout: 30
# Fail the job when it opened/updated tickets, so the workflow shows red while problems exist.
servicenow_fail_when_findings: true

# servicenow_health.yml: also check the MID Servers (needs read access to the ecc_agent table)
servicenow_check_mid: true
servicenow_slow_seconds: 5 # the API answering slower than this -> warning
```

poam (roles/poam/defaults/main.yml)

YAML

```
# POA&M STATUS: which Plan of Action & Milestones items are overdue or due soon, and (optional)
# which open CAT I / CAT II findings in STIG Manager have no POA&M item yet.
#
# The POA&M list is a CSV file in your work Git (export from eMASS or your spreadsheet, keep the
# columns below, save as CSV). POA&M data is sensitive: keep that repository access-controlled.
poam_csv: "{{ playbook_dir }}/../poam/poam.csv"

# Column names in YOUR file for each field this check needs (change the right-hand side).
poam_columns:
  id: POAM ID                # unique item number
  weakness: Weakness         # what is wrong
  severity: Severity         # CAT I / CAT II / CAT III, or High / Medium / Low
  status: Status             # Ongoing, Completed, Risk Accepted ...
  due: Scheduled Completion Date # a date
  owner: Owner               # person or office
  vuln_ids: Vulnerability IDs # STIG group IDs it covers (V-123456 ...), for the STIG
    Manager cross-check
poam_closed_statuses: [Completed, Closed, Risk Accepted, Cancelled]
poam_due_soon_days: 30
poam_date_formats: ['%Y-%m-%d', '%m/%d/%Y', '%m/%d/%y', '%d-%b-%Y', '%d %b %Y', '%b %d, %Y']

# ---- STIG Manager cross-check (optional) -----
# Needs a Keycloak client that may use the client-credentials grant, with the scope
# stig-manager:collection:read, and a grant in the collections to read. Your realm owner creates
# it - this repository never changes the realm. The "STIG Manager API" AAP credential then sets
# STIGMAN_TOKEN_URL, STIGMAN_CLIENT_ID and STIGMAN_CLIENT_SECRET.
poam_stigman_api: ""          # e.g. https://stigman.example.mil/api ; empty = skip
poam_stigman_collections: []  # collection names; empty = every collection the client
  can read
poam_stigman_severities: [high, medium] # CAT I and CAT II findings must have a POA&M item
# Scope to ask for. Empty = the client's default scopes (they must include
# stig-manager:collection:read or stig-manager:collection).
poam_stigman_scope: ""
poam_validate_certs: true
```

patch (roles/patch/defaults/main.yml)

YAML

```
# PATCH: install updates with dnf, reboot if the updates need it, and prove the host came back:
# every service that was running before must be running after. One host at a time by default;
# the rollout stops at the first host that fails.

patch_packages: ['*']                # what to update ('*' = everything that has an update)
patch_security_only: false            # true = only updates that fix security advisories
patch_exclude: []                    # never update these here, e.g. ['mariadb*', 'kernel*']

# Reboot: when_needed (needs-restarting says so) | never | always
patch_reboot: when_needed
patch_reboot_timeout: 1800            # seconds to wait for the host to come back
# Hosts in these groups are patched but NEVER rebooted by this playbook (reboot them by hand,
# in a maintenance window). AAP itself belongs here: a job must not reboot the AAP host it runs
# on.
patch_never_reboot_groups: [aap_hosts]
# Hosts in these groups are skipped entirely.
patch_never_patch_groups: [no_patch]

patch_min_free_mb: 1024               # free space needed in /var before starting
patch_service_wait: 120               # seconds to wait for services to come back after a
reboot                                #
# Services that come and go on their own and are not compared before/after (regular expression)
patch_ignore_services: '^ (user@|user-runtime-dir|session-|dnf-makecache|packagekit|fwupd|run-|sys
temd- (hostnamed|timedated|localed|coredump)) '
```

troubleshoot (roles/troubleshoot/defaults/main.yml)

YAML

```
# TROUBLESHOOT: the commands an experienced admin runs first for a kind of problem, run for you
# (read-only), each with a line saying what it shows. The matching health checks run too, so
# you also get plain-words findings. Pick the area; for "service" and "network" say what.
#
# ts_area:    overview | disk | performance | service | network | selinux | fapolicyd |
#             login | time | logs | mariadb
# ts_service: the systemd unit for area=service (e.g. nginx, mariadb, stigman-api)
# ts_target:  host:port for area=network (e.g. ldap.example.mil:636)
# ts_since:   how far back to read logs (journalctl --since), e.g. -2h, -1d, "2026-09-26 08:00"
ts_area: overview
ts_service: ""
ts_target: ""
ts_since: "-2h"
ts_output_lines: 60                # keep the last this-many lines of each command

# What each area runs. `why` is printed above the command's output. Commands must be read-only.
# ... (the rest of this file: see the repository)
```

service_watch (roles/service_watch/defaults/main.yml)

YAML

```
# ---- What to watch -----
# The podman containers on the host, in dependency order (a container's dependencies first).
#   name: the container name, as `podman ps -a` shows it
#   url: optional page to check as well, e.g. https://127.0.0.1/
#   unit: optional: the systemd unit that runs the container. Normally found by itself (below).
watch_containers:
  - name: stigman
  - name: nginx
watch_url_ok_status: [200, 301, 302, 401, 403]   # HTTP codes that count as "answering"

# ---- How each container is run (found by itself) -----
# A container that a systemd unit runs (a Quadlet .container file, `podman generate systemd`, or a
# unit you wrote) must be started with systemctl, never with `podman start`: the unit owns it, and
# while the unit is stopped the container usually does not exist at all (Quadlet removes it).
# For each container the role looks for its unit, in this order: `unit:` above; the container's
# PODMAN_SYSTEMD_UNIT label; a Quadlet file in watch_quadlet_dirs that names it (ContainerName=,
# or the default name systemd-<file name>); a unit NAME.service or container-NAME.service that
# runs podman. No unit: it is a plain podman container, started with `podman start NAME`.
watch_find_units: true
watch_quadlet_dirs: [/etc/containers/systemd, /usr/share/containers/systemd]

# ---- What happens when one is down -----
# approval: ACT only diagnoses. Its proposed fix waits for the AAP approval step, then
#             service_fix_approved.yml runs exactly the approved command(s) - no model involved.
#             When ACT proposes nothing (or cannot run), the standard fix is proposed instead:
#             `systemctl start UNIT` for a container a unit runs, `podman start NAME` otherwise.
# self-heal: ACT may start/restart the watched containers (through their units) by itself, and
#             nothing else. Still down after that: the playbook runs the standard fix itself.
#             Any other fix goes to a person, as in approval mode.
watch_mode: approval
watch_verify_retries: 6                # re-checks after a fix (every watch_verify_delay
seconds)
watch_verify_delay: 5
```

YAML (continued)

```
# ---- Where incidents are recorded (on the host) -----
# One JSON line per event, plus a syslog line (tag service-watch) that your log forwarding
# (Splunk, Elastic, rsyslog relay) picks up. AAP's job history keeps its own copy.
watch_record_file: /var/log/service-watch/incidents.jsonl
watch_record_syslog: true

# ---- ACT settings -----
# Which provider and model: site_act_provider / site_act_model / site_act_url / site_act_ca, the
# same settings every runbook uses (roles/site_act/defaults/main.yml). The key comes from the
# "ACT model key" AAP credential, never from here. Extra ACT settings for service watch only:
watch_act_env: {}
watch_act_timeout: 600 # seconds for one ACT run

# What ACT is told, on top of the act_triage role's standard instructions.
watch_instructions: |-
  The watched podman containers on this host, in dependency order (start earlier ones first).
  They run as root (rootful podman). How each one is run, and the ONLY right way to start it:
  {{ _watch_how_text }}
  A container that a systemd unit runs must never be started with `podman start` or `podman run`:
  the unit owns it, and while the unit is stopped the container usually does not exist at all
  (Quadlet removes it), so "no such container" is normal then, not the problem.
  For each finding, find the root cause with read-only commands:
  `systemctl status UNIT --no-pager` and `journalctl -u UNIT --since -2h --no-pager` (for a
  container a unit runs: why the unit stopped or failed),
  `podman ps -a` (every container and its state),
  `podman inspect NAME` (State: Status, ExitCode, FinishedAt, OOMKilled, Health),
  `podman logs --tail 80 NAME` (its last output before it stopped),
  `journalctl -t podman --since -2h --no-pager` (podman events, e.g. who stopped it and when),
  and the logs of the containers it depends on if those look unhealthy.
  Say whether it crashed, was stopped on purpose, or failed for another reason (configuration,
  a dependency, a port, disk space). Quote the lines that show it.
  Starting it again the right way (above) is the fix when it stopped or crashed and nothing else
  is wrong. If a container it depends on is down, fix that one first. If the cause is something
  a start cannot fix, say so and give the exact fix for a person to approve.
```

YAML (continued)

```
{% if watch_mode == 'self-heal' %}You may start or restart the watched containers yourself, the
right way (above), and change nothing else: propose any other change, do not make it. Verify
with `podman inspect NAME` and expect_contains `"Running": true`.{% endif %}
# ... (the rest of this file: see the repository)
```

stigman_stack (roles/stigman_stack/defaults/main.yml)

YAML

```
# STIG Manager stack on podman, managed by systemd through Quadlet unit files:
# stigman-mysql (MySQL 8.4) <- stigman-api (STIG Manager) <- stigman-nginx (TLS, port 443)
# Keycloak is EXTERNAL: the realm already exists and this role never touches it.
# Needs podman >= 4.4 (Quadlet): RHEL 9.2+ / RHEL 8.8+.

# system = rootful (/etc/containers/systemd, systemctl) <- production
# user = rootless (~/.config/containers/systemd, systemctl --user) <- tests / rootless hosts
stigman_scope: system

stigman_images:
  mysql: docker.io/library/mysql:8.4
  api: docker.io/nuwcdivnpt/stig-manager:1.6.17 # pin a version; bump deliberately
  nginx: docker.io/library/nginx:stable

stigman_config_dir: /etc/stigman # nginx.conf, TLS files, CA bundle
stigman_https_port: 443 # host port nginx publishes
stigman_server_name: "{{ inventory_hostname }}"

# Existing Keycloak realm (server-side URL used by the API; browser-side URL for the web client)
stigman_oidc_provider: "https://keycloak.example.mil/realms/stigman"
stigman_client_oidc_provider: "{{ stigman_oidc_provider }}"
stigman_client_id: stigmanager
stigman_classification: U # banner: U, CUI, C, S, TS, SCI, NONE

stigman_db_schema: stigman
stigman_db_user: stigman
stigman_mysql_buffer_pool: 1G # STIG Manager docs: >= 8G for real
deployments

# How long to wait for the API to answer through nginx after (re)starting
stigman_ready_retries: 40
stigman_ready_delay: 6

# ---- Secrets: NEVER set these in a vars file or inventory -----
```

YAML (continued)

```
# They come from AAP credentials (aap/credential_types/*.yaml) as environment variables / files
# on the controller (execution environment). See docs/SECRETS.md.
stigman_mysql_root_password: "{{ lookup('ansible.builtin.env', 'STIGMAN_MYSQL_ROOT_PASSWORD') }}"
stigman_db_password: "{{ lookup('ansible.builtin.env', 'STIGMAN_DB_PASSWORD') }}"
stigman_tls_cert_src: "{{ lookup('ansible.builtin.env', 'TLS_CERT_FILE') }}"
stigman_tls_key_src: "{{ lookup('ansible.builtin.env', 'TLS_KEY_FILE') }}"
stigman_tls_ca_src: "{{ lookup('ansible.builtin.env', 'TLS_CA_FILE') }}"
```